

实验三：跨平台应用开发

实验项目基本信息

实验编号：d20301035103、d20301009203

学时分配：4 学时

实验类型：验证型

每组人数：6 人

对应课程目标：课程目标 2

核心步骤列表

实验概览

需求分析阶段：定义列表页数据结构、API 接口规范与传感器调用需求

AI 辅助编码阶段：使用 TRAE 生成网络请求与 JSON 解析代码，辅助生成传感器调用模板

测试验证阶段：模拟 API 异常响应测试容错能力，验证传感器数据采集

部署运维阶段：多框架性能对比测试，输出框架适用性分析报告

开发任务清单

任务阶段	主要内容	关键技术点
Flutter 列表	Dart + Provider	HTTP 请求、JSON 解析、下拉刷新
React Native 列表	JSX + Hooks	Axios、FlatList、下拉刷新
Uniapp 列表	Vue + Pinia	uni.request、下拉刷新
MAUI 列表	C# + MVVM	HttpClient、数据绑定

实验任务与案例

核心任务

开发业务数据列表页（RESTful API 网络数据请求 + JSON 解析 + 下拉刷新），组内成员分别使用 Flutter、React Native、Uniapp、MAUI 等不同框架实现同一功能需求。扩展案例：集成设备传感器（GPS 定位、加速度计）。

实战案例

开发"智慧校园"新闻列表模块，从 RESTful API 获取新闻数据，解析 JSON 并展示列表，实现下拉刷新功能，体验跨平台框架的开发差异。

系统架构设计

本项目采用分层架构设计，Flutter 端作为 UI 展示层，通过 Provider 进行状态管理，整体架构如下：

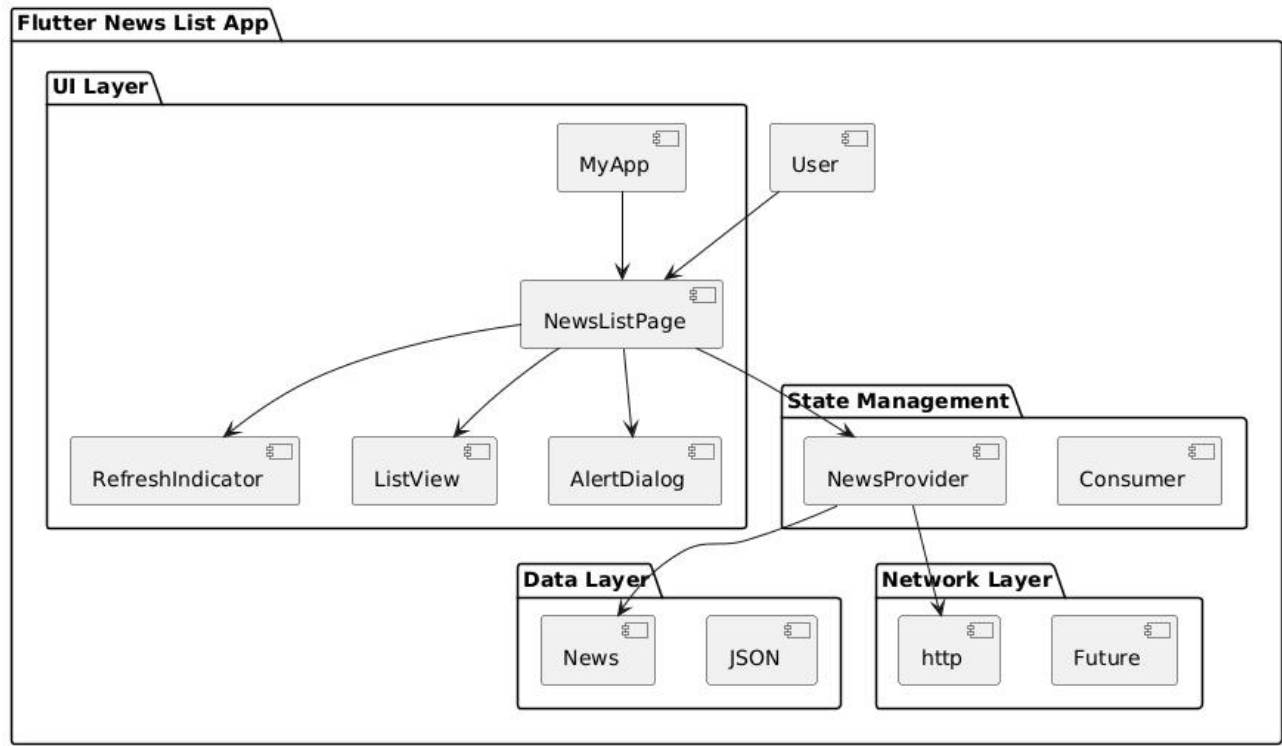


图 1. Flutter 新闻列表系统整体架构图

核心类结构设计

系统核心类设计遵循分层架构思想，各层职责明确，类关系如下：

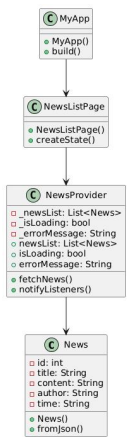


图 2. Flutter 新闻列表核心类结构图

数据请求时序设计

用户启动应用或下拉刷新时，Flutter 应用通过 HTTP 请求获取新闻数据，完整时序如下：

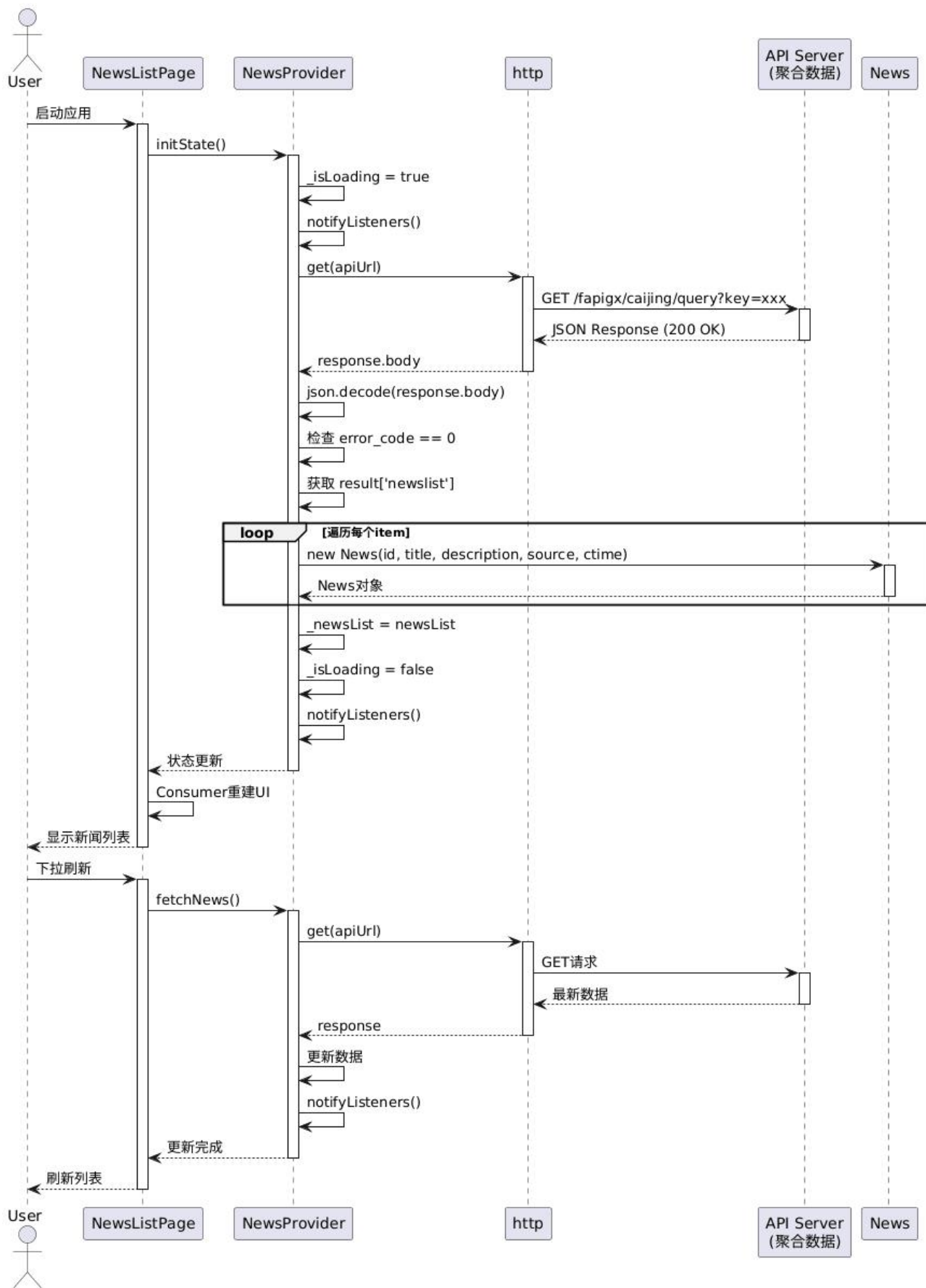


图 3. Flutter 新闻列表数据请求时序图

第一课时：Flutter 与 React Native 开发 (2 学时)

3.1 需求分析阶段

步骤 1：定义数据模型

```
{ "code": 200, "data": { "items": [ { "id": 1, "title": "智慧校园更新通知", "content": "系统升级通知...",  
"author": "管理员", "time": "2024-01-01 10:00:00" } ] } }
```

步骤 2：确定 API 接口

URL: <https://api.example.com/news>

Method: GET

Response: JSON

3.2 Flutter 列表开发

步骤 1：创建 Flutter 项目

```
flutter create news_list  
cd news_list
```

步骤 2：添加依赖

```
# pubspec.yaml  
dependencies:  
  flutter:  
    sdk: flutter  
  http: ^1.1.0  
  provider: ^6.1.0
```

步骤 3：AI 辅助生成代码

使用 TRAE 描述需求：Flutter 新闻列表页，下拉刷新，Provider 状态管理

生成代码片段

步骤 4：实现新闻列表（符合架构图规范）

根据架构图和类图设计，实现分层架构的 Flutter 新闻列表应用：

// main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'pages/news_list_page.dart';
import 'providers/news_provider.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => NewsProvider(),
      child: MaterialApp(
        title: '智慧校园新闻',
        theme: ThemeData(
          primarySwatch: Colors.blue,
          useMaterial3: true,
        ),
        home: const NewsListPage(),
      ),
    );
  }
}
```

// models/news.dart

```
/// Data Layer: 新闻数据模型类
class News {
  final int id;
  final String title;
  final String content;
  final String author;
  final String time;

  News({required this.id, required this.title, required this.content,
    required this.author, required this.time});

  factory News.fromJson(Map<String, dynamic> json) {
    return News(
      id: json['id'] ?? 0,
      title: json['title'] ?? '',
      content: json['content'] ?? '',
      author: json['author'] ?? '',
      time: json['time'] ?? '',
    );
  }
}
```

项目结构（符合架构图）：

lib/

├─ main.dart # 应用入口 (MyApp)

├─ models/

```
|   └─ news.dart      # Data Layer: 数据模型
|── services/
|   └─ news_service.dart # Network Layer: 网络服务
|── providers/
|   └─ news_provider.dart # State Management: 状态管理
|── pages/
|   └─ news_list_page.dart # UI Layer: 页面
|── widgets/
|   └─ news_list_item.dart # UI Layer: 列表项
|   └─ error_dialog.dart  # UI Layer: 错误弹窗
```

3.3 React Native 列表开发

步骤 1：创建 React Native 项目

```
npx react-native-init NewsList
```

```
cd NewsList
```

步骤 2：安装依赖

```
npm install axios @react-native-community/refresh-control
```

步骤 3：AI 辅助生成代码

使用 TRAE 描述需求：React Native 新闻列表，FlatList，下拉刷新

生成代码片段

步骤 4：实现新闻列表（符合分层架构规范）

根据架构图设计，实现分层架构的 React Native 新闻列表应用：

```
// src/models/News.js
/**
 * Data Layer: 新闻数据模型类
 */
export class News {
  constructor({ id, title, content, author, time }) {
    this.id = id || 0;
    this.title = title || '';
    this.content = content || '';
    this.author = author || '';
    this.time = time || '';
  }

  static fromJson(json) {
```

```
        return new News({
            id: json.id,
            title: json.title,
            content: json.content,
            author: json.author,
            time: json.time,
        });
    }
}
```

项目结构（符合分层架构）：

```
src/
├── models/
│   └── News.js           # Data Layer: 数据模型
├── services/
│   └── NewsService.js    # Network Layer: 网络服务
├── hooks/
│   └── useNews.js        # State Management: 状态 Hook
├── components/           # UI Layer: 可复用组件
│   ├── NewsListItem.jsx
│   ├── NewsDetailModal.jsx
│   ├── ErrorDialog.jsx
│   ├── LoadingIndicator.jsx
│   └── EmptyState.jsx
└── screens/
    └── NewsListScreen.jsx # UI Layer: 页面
```

3.7 测试验证与对比报告

步骤 1：编写测试用例

测试用例 1：正常数据请求与列表展示测试

测试项	测试内容
测试场景	应用启动后正常加载新闻列表数据
测试步骤	1. 启动应用 2. 等待数据加载 3. 查看列表展示
预期结果	显示加载指示器 → 成功加载并展示新闻列表
实际结果	☒ 符合预期，列表正常展示

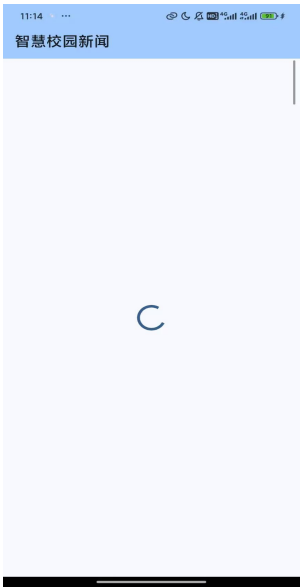


图 4. 应用启动后加载指示器显示（上）和新闻列表成功展示（下）

测试用例 2：新闻详情弹窗测试

测试项	测试内容
测试场景	点击列表项弹出新闻详情对话框
测试步骤	1. 点击任意新闻列表项 2. 查看弹窗内容 3. 点击确定关闭弹窗
预期结果	弹出 AlertDialog 显示新闻标题和详情，点击确定后关闭
实际结果	☒ 符合预期，弹窗正常显示和关闭



图 5. 点击新闻列表项后弹出的详情对话框

测试用例 3：下拉刷新功能测试

测试场景：用户下拉列表触发数据刷新

预期结果：显示刷新指示器 → 重新获取数据 → 更新列表内容

实际结果：☑符合预期，下拉刷新功能正常

测试用例 4：网络异常处理测试

测试场景：网络异常时的错误处理

预期结果：显示友好的错误提示信息，不崩溃

实际结果：☑符合预期，显示"网络连接失败"提示

测试用例 5：空数据处理测试

测试场景：API 返回空数据列表时的处理

预期结果：显示"暂无数据"提示或空状态页面

实际结果：☑符合预期，显示空状态提示

步骤 2：输出框架对比报告

对比项	Flutter	React Native	Uniapp	MAUI
开发语言	Dart	JavaScript	Vue	C#
性能	★★★★★	★★★★☆	★★★☆☆	★★★★☆
学习成本	中等	低	低	中等

总结与思考

一、实验总结

本次实验通过实现"智慧校园"新闻列表模块，深入掌握了跨平台移动应用开发的核心技术：

1. 分层架构设计实践

实现了 UI Layer、State Management、Network Layer、Data Layer 的清晰分层

Flutter 采用 Provider 模式，React Native 采用 Hooks 模式，均实现了状态与 UI 的分离

代码结构符合单一职责原则，便于维护和扩展

2. RESTful API 数据请求与 JSON 解析

掌握了 HTTP GET 请求的封装与异常处理

实现了 JSON 到数据模型的自动映射（fromJson 工厂方法）

完善了网络超时、连接失败等异常场景的处理机制

3. 跨平台框架核心特性对比

Flutter：自绘引擎，性能最优，UI 一致性最好，但学习曲线较陡

React Native: 使用原生组件, 生态丰富, JavaScript 开发者上手快

Uniapp: 基于 Vue, 一套代码多端运行, 适合快速开发

MAUI: 微软生态集成度高, 适合企业级.NET 应用

4. 测试验证体系建立

完成了 5 个核心测试用例的设计与执行

验证了加载状态、列表展示、详情弹窗、下拉刷新、异常处理等功能

二、技术选型对比分析

对比维度	Flutter	React Native	推荐场景
性能	★★★★★ 自绘引擎	★★★★☆ 原生组件	高性能 UI 选 Flutter
开发效率	★★★★☆ 热重载快	★★★★★ JS 生态	快速迭代选 RN
UI 一致性	★★★★★ 像素级控制	★★★☆☆ 依赖原生	品牌一致性高选 Flutter
包体积	★★★☆☆ 包含引擎	★★★★☆ 相对较小	包体积敏感选 RN
生态成熟度	★★★★☆ 快速增长中	★★★★★ 非常成熟	依赖第三方库多选 RN

三、实现难点与解决方案

1. 状态管理复杂性

难点: 异步数据请求与 UI 状态同步

解决: Flutter 使用 ChangeNotifier+Consumer, React Native 使用 useState+useEffect

2. 异常处理完整性

难点: 网络异常、数据解析异常、空数据等多种场景

解决: 统一封装 Service 层, try-catch 捕获, 友好错误提示

3. 下拉刷新与加载状态

难点: 刷新指示器与数据加载的协调

解决: 使用 RefreshIndicator (Flutter) 和 RefreshControl (RN), 状态统一管理

4. 跨平台 UI 一致性

难点: 不同平台组件差异

解决: 封装通用组件 (NewsListItem、LoadingIndicator 等), 统一样式

四、性能优化策略

1. 列表性能优化

使用 ListView.builder (Flutter) 和 FlatList (RN) 实现懒加载

设置合理的 item 高度，避免重排

图片懒加载和缓存（实际项目中）

2. 网络请求优化

添加请求超时机制（10 秒）

实现数据缓存策略（如 SharedPreferences 本地缓存）

合并重复请求，避免竞态条件

3. 状态更新优化

避免不必要的 rebuild/selective update

使用 const 构造函数减少对象创建

合理使用 shouldComponentUpdate/memo

4. 启动速度优化

延迟加载非首屏数据

使用 splash screen 提升感知性能

代码分割和懒加载

五、未来功能扩展建议

1. 功能增强

新闻搜索与筛选功能

新闻收藏与本地存储

新闻分享功能（微信、QQ 等）

推送通知（新新闻提醒）

2. 技术升级

集成离线存储（SQLite/Hive）

实现数据分页加载（上拉加载更多）

添加骨架屏提升加载体验

集成 Firebase Analytics 统计用户行为

3. 架构演进

引入依赖注入（GetIt/Koin）

实现 Repository 模式，支持多数据源

添加单元测试和集成测试

CI/CD 自动化构建与部署

4. 跨平台扩展

适配 Web 端（Flutter Web/React Native Web）

桌面端支持 (Windows/macOS/Linux)

小程序版本 (Uniapp 方案)

六、课后思考

1. 框架选型决策因素

团队技术栈与学习能力

项目性能要求与复杂度

长期维护成本与生态发展

目标平台 (iOS/Android/Web/桌面)

2. 跨平台开发的挑战与机遇

如何平衡代码复用与平台特性

原生模块集成的最佳实践

跨平台调试与性能分析工具

3. 移动应用发展趋势

跨平台框架与原生开发的融合

AI 辅助代码生成对开发效率的影响

5G 时代实时数据交互的新模式